



AI_Project 控制台 · 經營層說明

給軟體公司決策者

任務在系統裡， 交付在本機。

工作單在 Jira、GitHub，程式在工程師電腦，AI 會改碼卻看不到現場。中間缺一座指揮台：把治理、課責、堆疊真相，接到你們已有的 Agent。

現況

你們並不缺工具。 缺的是編成一套日常。



專案只寫在系統裡

工作單開了，現場卻對不上分支、編譯與驗收。PM 看到的狀態，工程師螢幕上不一定是同一件事。



人在表上

工時在試算表、任務在另一個系統、報帳靠回憶。專案毛利最後是猜的。



AI 與現場脫節

買了 Cursor 或 Copilot，Agent 仍在猜連接埠、猜有沒有編過。改完碼沒人用同一套真相驗證，也沒有審計。

不接起來的代價

隱形成本不會出現在損益表標題。



毛利是猜的

工時匯不出、對不上 Issue，專案結算只能事後拼湊。

驗收沒有軌跡

客訴時講不清誰在何時放了什麼、關卡有沒有擋住。

新人靠口耳相傳

啟動順序、需重編、求救上下文，都活在資深同事的腦子裡。

圖為示意，用來對照痛點，不是客戶實測。

角色定位

本機交付作業系統，接上你們的 Agent。

它不是

- ✗ 另一套雲端專案管理，用來取代你們現有的任務系統
- ✗ 取代 Cursor、Copilot 或任何寫程式的 Agent
- ✗ 把原始碼送到控制台自己的雲端代管

它是

- + 坐在 Git、GitHub、本機 Agent 之上的 Agentic 指揮層
- + 讓 Agent 用堆疊真相做事：編譯、啟停、查 Log，並留下審計
- + 本機桌面產品：Agent 跑在公司電腦，真相留在倉裡

經營層

AI 授權要換成可驗收的交付；工時對得上，Agent 動作有審計。

專案經理

任務可請 Agent 協助；驗收與報帳關卡仍由人擋。

Tech Lead

Agent 用 MCP 查就緒與需重編，不是猜。政策擋得住危險操作。

工程師

求救帶齊上下文；專案問答在控制台內，不佔編譯。

產品地圖

同一個視窗，四層能力。

同一套 Git 帳號。Agent 是操作者，控制台是真相來源。開發走編譯與承接；需求走治理與生命週期。

01 交付作業

讓堆疊真話

掃描、需重編、一鍵啟停、
Log、文件。摘要列一眼看值班
狀態。

02 專案治理

讓任務可驗收

新需求與設計變更分開。沒有
驗收條件不能發 Issue；CI 未綠
不能合。

03 人員課責

讓人對得上帳

GitHub 帳號就是人。接受任
務、工時儀表、匯出報帳單並
對上 Issue。

04 Agentic

讓 AI 閉環

本機 Agent 改碼；MCP 回查編
譯與堆疊。求救帶上下文，右
側有審計。

從進件到報帳，關卡寫在產品裡。

草稿

設計就緒

已拆解

已發出

實作中

驗證中

可審查

入主線

已發行

已部署

已驗收

已報帳

新需求 ≠ 設計變更

從無到有走 REQ；改既有行為走 ECR，要填現況、期望、影響，並上傳分析或設計文件。介面還要草圖或剪圖。

擋得住的關卡

沒有驗收條件不能發 Issue。CI 未綠不能合併。驗收要勾完條件，相關 PR 已入主線，且已部署或明示不部署。

暫停與收回留紀錄

已發出的進件不能直接刪。暫停與收回會留言通知承辦人，清單留痕跡。已入主線的程式要另開變更還原。

人的主鍵是 GitHub 帳號，不是另一張卡。

任務歸戶

開發台看「我的+未指派」，一鍵接受。需求看板看全部未關閉與指派，含他人。

工時自動切

從開啟專案起算。切換或關閉就切開時段。滿 8 小時變黃、滿 10 小時變紅。

週月季年

儀表板看各專案時數與作息；同一 GitHub 倉會併在一起，帳號不會混。

報帳對得上

匯出 Markdown 與 CSV。需求台匯入後，對上已發出的 Issue 編號，標成已報帳。

尚未登入時先記在本機使用者；登入後把未歸戶時段併到該 GitHub 帳號。匯出時會提醒負責人。

Agent 負責改碼，控制台負責真相。

不取代你們已買的 AI。控制台組好上下文，交給本機後端；Agent 也可回呼控制台驗證，而不是猜。

後端可換

預設 Cursor，也可 Claude Code、Codex、Aider、VS Code、Windsurf 或自訂命令。都在公司電腦上跑。

帶上下文求救

畫面卡關、服務 Log、編譯失敗、指派任務：各帶對應材料給 Agent。文件與提交說明也可請 AI 起草。

MCP 回呼

Agent 可查值班摘要、編譯過期項目、讀 Log、啟停服務。修完要驗證，不是只改檔。

審計與問答

MCP 成功或被拒都入對帳單。專案問答在控制台內只讀堆疊，不外開 IDE，也不鎖編譯。

差異

補上執行層，不是再買一套任務系統。

能力	雲端 PM	只用 AI IDE	自製腳本	本產品
任務與生命週期	強	弱	無	接到 GitHub，關卡在本機
本機堆疊真相	無	弱	部分	掃描、需重編、就緒
工時對帳	外掛	無	試算表	內建，對上 Issue
Agentic 閉環	無	會改碼	無	求救 + MCP 驗證 + 審計
程式碼是否經本產品雲端	視供應商	視模型	否	否

為什麼現在採購

用一套本機產品，收斂四件事。

90 分鐘導入

不是半年顧問案。Tech Lead 用自己的倉走完掃描到求救。

一張對帳單

工時、已關閉 Issue、開啟或合併的 PR，匯出給負責人。

驗收能擋關

毛利與客訴都需要軌跡。關卡寫在產品裡，不是寫在 wiki。

Agent 有閉環

已有的 AI 繼續用。控制台讓它接到堆疊真相，動作看得到、擋得住。



雷達圖為能力涵蓋示意，用來對照角色，不是評分排名。

決策時會問的

四個疑慮，直接回答。

我們已經有 Cursor

繼續用。本產品不取代 IDE。它讓 Agent 接到需重編、就緒與 Log，並用 MCP 驗證；右側看得到呼叫與拒絕。

我們已經有任務系統

Jira、Azure DevOps 可以繼續用。本產品補本機真相、關卡與工時對帳。任務面目前接 GitHub Issue。

程式碼會不會上雲

控制台是本機桌面，不把原始碼上傳到自己的雲。Agent 是你們選的本機後端。MCP 有審計。

會不會被鎖死

真相在 Git 與遠端倉。離開仍帶走 Issue、PR、文件與程式。後端可換，不是綁一家 AI。

規劃 · 尚未出貨

軟體公司還要一層營運，寫在規格書裡。

工程師繼續用本機控制台。主管、人資、外包窗口用客戶自架的 Blazor Web App。不要把下面六塊講成今天能打開。

人員與外包

正職、個人外包、承攬派駐。窗口只能看己方；費率不是另一張 GitHub 帳號就結束。

派工全貌

GitHub 仍是任務面。公司看誰超載、誰能派。寫回 assignee 可開可關。

專案與客戶

客戶、合約、階段、甘特。分析師與工程師來自派工，不另養一份名單。

預算與毛利

收入對人事成本。不是總帳，是專案還能不能做。

薪資三種規則

控制台畫狀態並上傳。按月、時計、按專案彙總；人資核准後匯出。

戰情室

經營層一屏：延期、無人接、毛利紅燈。不是工程師的第二個看板。

權威稿：產品規格。簡報這一頁只為標示「規劃」，避免買家以為安裝包已有戰情室。

下一步

用你們自己的倉，跑 90 分鐘。

建議試用路徑（現況）

指定一位 Tech Lead、一位專案經理

1. 安裝 Windows 包，打開一份真實多服務倉。
2. 看「需重編」、編譯過期項目、按啟動。
3. 登入 GitHub，看任務與工時徽章。
4. 用需求台走一筆進件到發出（或看既有 Issue）。
5. 卡住時走一次求救；若已接 MCP，請 Agent 用值班摘要驗證，不要猜。

今天就能做

從 Releases 下載安裝包，不必編譯本產品原始碼。

成功標準

Agent 不再猜現場；工時對得上人；驗收留得住紀錄。

要營運層

會後讀產品規格。那是規劃，不是這一小時能點開的畫面。